SPACL: Speculative Parallelism and Conflict Learning for Scalable OWL Ontology Reasoning

Anusorn Chaikaew[a,1], Varin Chouvatut[a,2], Ekkarat Boonchieng[a,3,*]

[a]*Chiang Mai University, Department of Computer Science, Faculty of Science, Chiang Mai, 50200, Thailand*

Abstract

We present SPACL (Speculative Parallel Tableaux with Adaptive Conflict Learning), a novel ALC/SHOIQ reasoner that achieves significant performance improvements through speculative parallelism combined with conflict-driven learning. To the best of our knowledge, SPACL is among the first open implementations to combine work-stealing parallelism with nogood learning for this setting, addressing the challenge of exponential search spaces in tableau-based reasoning.

Our key contributions include: (1) a speculative parallel tableaux algorithm that explores branches concurrently using work-stealing; (2) an adaptive threshold mechanism that automatically selects between sequential and parallel processing based on problem complexity, achieving 18–73× speedup over naive parallelization for small ontologies; (3) thread-local nogood caching that reduces synchronization overhead by 80%; (4) a global thread pool optimization reducing overhead by 2.1×; and (5) a production-quality Rust implementation with comprehensive parser support for RDF/XML, OWL2 Functional, Manchester, and JSON-LD syntaxes. SPACL demonstrates 5× speedup at 10,000 classes while maintaining $< 2\times$ overhead for small ontologies.

Benchmarks demonstrate **40–707× wall-clock speedup** on small disjunctive ontologies (<1,000 classes) vs HermiT; however, this includes significant JVM startup overhead (~3–4s). For pure reasoning time (excluding JVM startup), algo-

*Corresponding author. Tel.: +66-5394-3413

Email addresses: `anusorn_chaikaew@cmu.ac.th` (Anusorn Chaikaew), `varin.ch@cmu.ac.th` (Varin Chouvatut), `ekkarat.boonchieng@cmu.ac.th` (Ekkarat Boonchieng)

[1]First author

[2]Second author

[3]Corresponding author

rithmic speedup is 15–30×. In our latest large-suite head-to-head run (`head2head_round4_large_full_`
SPACL records the best wall-clock result on all five tested ontologies (chebi, doid, go-basic, pato, uberon) under an identical 900s timeout protocol. This improvement is driven by a full-fidelity structural RDF/XML parser and optimized materialization path: stage-level A/B runs show parse-only reductions of 96.96–97.98% on doid/go-basic/uberon relative to the previous structural baseline.

Keywords: ALC/SHOIQ, Tableaux Reasoning, Parallel Algorithms, Nogood Learning, Description Logics, Semantic Web

---

1. Introduction

Description Logics (DLs) provide the formal foundation for the Semantic Web, with OWL2 DL serving as the W3C standard ontology language [1]. Tableaux algorithms remain a widely used approach for OWL2 DL reasoning, offering completeness and soundness for complex TBox and ABox reasoning tasks [2]. However, the exponential nature of tableau expansion creates significant performance challenges for large-scale ontologies. This work focuses on the ALC/SHOIQ fragment, which forms the basis of most OWL2 ontologies and captures the core challenges of disjunctive reasoning.

While modern hardware provides abundant parallel processing capabilities, existing DL reasoners have not effectively exploited this potential. Commercial reasoners like Konclude provide parallel reasoning but remain closed-source [3], limiting reproducibility and extension. Open-source alternatives like Pellet [4], its actively maintained fork Openllet [5], and HermiT [6] rely primarily on sequential algorithms. Furthermore, parallel exploration of search spaces without learning from failures leads to redundant computation—a missed optimization opportunity.

1.1. Research Questions

This work addresses the following research questions:

RQ1: Can speculative parallelism improve DL reasoning performance without excessive overhead?

RQ2: How can nogood learning from failed branches reduce search space?

RQ3: What is the optimal threshold for switching between sequential and parallel processing?

RQ4: Can these techniques be combined in a production-quality implementation?

1.2. Contributions

We address these challenges through SPACL, which makes five novel contributions:

C1: **Speculative Parallelism with Work-Stealing**: SPACL employs a work-stealing scheduler that dynamically distributes tableau branches across worker threads. When a disjunction is encountered, both branches are speculatively explored in parallel, with workers stealing tasks from each other to maintain load balance (Section 4.2).

C2: **Conflict-Driven Nogood Learning**: When a branch leads to contradiction, SPACL learns a "nogood" clause recording the conflicting assertions. These nogoods prune future branches that would lead to the same contradiction, significantly reducing the search space. Thread-local caching minimizes synchronization overhead while maintaining learning effectiveness (Section 4.3).

C3: **Adaptive Parallelism Threshold**: Rather than always using parallel processing (which incurs overhead), SPACL automatically selects the optimal strategy based on estimated problem complexity. Small ontologies use sequential processing; large ontologies benefit from parallel exploration. Our cost-based model achieves 18–73× speedup over naive parallelization for small cases (Section 4.4).

C4: **Global Thread Pool Optimization**: We replaced per-call thread spawning with a global rayon thread pool, reducing parallelization overhead from 235μs to 112μs (2.1× improvement) for simple disjunctions (Section 6.5).

C5: **Comprehensive Parser Suite**: We provide production-quality parsers for RDF/XML, OWL2 Functional Syntax, Manchester Syntax, and JSON-LD, enabling interoperability with existing ontology engineering tools and supporting complex expression constructs including ObjectUnionOf, ObjectIntersectionOf, and ObjectComplementOf (Section 5.4).

Our implementation in Rust demonstrates these contributions translate to practical performance gains: 5× speedup at 10,000 classes, sub-microsecond per-class processing, and $< 2\times$ overhead even for small ontologies. SPACL achieves 26.2 million operations per second—orders of magnitude faster than Java-based alternatives.

1.3. Paper Organization

The remainder of this paper is organized as follows. Section 2 reviews related work in parallel reasoning and conflict learning. Section 4 presents the SPACL

algorithm and its components. Section 5 describes our Rust implementation. Section 6 presents comprehensive benchmarks including scalability, thread scaling, and comparison with existing reasoners. Section 7 concludes with limitations and future directions.

2. Research Relevance and Motivation

2.1. Fundamental Challenges in OWL2 DL Reasoning

OWL 2 DL (Web Ontology Language 2 Description Logic) reasoning remains fundamental to semantic web applications, knowledge graph construction, and biomedical informatics [7]. However, tableau-based algorithms face severe scalability challenges due to the exponential nature of tableau expansion, particularly for large-scale ontologies with complex axioms [8].

2.1.1. Exponential Search Space and Non-Determinism

The primary obstacle to scalable OWL2 DL reasoning is the exponential growth of the tableau search space. Faddoul and MacCaull [9] discuss non-determinism as a major source of complexity in expressive description logics, where disjunctive axioms can multiply the number of branches. This exponential branching behavior means that even moderately sized ontologies can generate large numbers of potential tableau branches, overwhelming sequential reasoners.

2.1.2. Underutilization of Parallel Hardware

Despite widespread availability of multi-core processors, the majority of open-source OWL reasoners continue to employ fundamentally sequential algorithms [10]. Quan and Haarslev [10] observe that while commercial reasoners offer some parallel reasoning capabilities, they remain closed-source, and many widely-used open-source alternatives have not been designed to exploit thread-level parallelism. This represents a significant missed opportunity, as parallel exploration of independent tableau branches could theoretically achieve near-linear speedups on modern hardware.

2.1.3. Redundant Computation Without Learning

A critical inefficiency in parallel tableau reasoning is the repeated exploration of branches that lead to identical contradictions [11]. Traditional tableau reasoners lack mechanisms to record and propagate learned conflicts across branches, resulting in exponentially redundant work [12].

2.2. Existing Approaches and Their Limitations

2.2.1. Shared-Memory Thread-Level Parallelization

Quan and Haarslev [10, 13] developed parallel shared-memory architectures that distribute independent subsumption tests across multiple threads and report substantial wall-clock improvements for classification tasks. However, these approaches focus primarily on classification rather than full ABox-intensive reasoning, and they do not modify the inner tableau logic to enable conflict learning or nogood reuse across parallel branches [13].

2.2.2. Distributed Materialization Frameworks

Distributed computing frameworks leverage Apache Spark and MapReduce to materialize OWL closures over large datasets. Liu and McBrien [14] introduced SPOWL, which compiles TBox axioms into iterative Spark programs. Gu et al. [15] developed Cichlid, and Benítez-Hidalgo et al. [16] presented NORA, combining Apache Spark with NoSQL databases to scale materialization to very large ABoxes.

Literature verification note (LIT-TODO-1): Reported gains in distributed materialization studies vary by dataset, profile, and execution environment; we therefore treat these papers as evidence of scalability direction rather than directly comparable end-to-end speedups for full OWL2 DL reasoning.

While distributed materialization achieves excellent scalability for rule-based fragments (OWL RL, OWL Horst, RDFS), these approaches are less applicable to full OWL2 DL sound-and-complete tableau procedures, which require handling non-determinism and backtracking [17].

2.2.3. Hybrid Partitioning Strategies

Wang et al. [18] introduced ComR, which identifies a minimal non-EL subontology and delegates much of the workload to an OWL EL reasoner. Steigmiller and Glimm [11] explored dynamic splitting of model construction with shared derivation caches. However, partitioning requires careful identification of the minimal non-EL portion and may be less effective when many axioms fall outside tractable fragments [18].

2.3. Conflict-Driven Learning: An Untapped Opportunity

2.3.1. Success in SAT/CSP Domains

Conflict-driven clause learning (CDCL) has revolutionized Boolean satisfiability (SAT) solving, enabling modern SAT solvers to handle instances with millions of variables [19]. Glorian et al. [20] developed NACRE, a nogood-and-

clause reasoning engine for constraint solving with data structures optimized for learning strategies, achieving competitive performance in CSP competitions.

2.3.2. The Research Gap in DL Reasoning

Despite the success of CDCL in SAT/CSP domains, our survey did not find clear open-source reports integrating conflict-driven nogood learning into tableau-based ALC/SHOIQ reasoners. While Liu et al. [21] applied deep learning to discover inference rules across ontologies, this work focuses on rule augmentation rather than tableau search optimization. Algahtani [22] demonstrated massively parallel hypothesis evaluation for inductive learning (MP-HTHEDL), but this targets ILP-style evaluations rather than tableau reasoning.
Literature verification note (LIT-TODO-2): This gap statement is bounded to publicly documented and open-source implementations covered by our survey; closed-source, unpublished, or non-indexed systems may exist outside this review scope.

The surveyed literature suggests a gap: parallel frameworks exist for OWL reasoning [10, 13, 14, 15] and nogood/clause learning machinery is well-established for constraint problems [20], but integrated adaptive conflict-driven parallel tableau implementations for ALC/SHOIQ are not commonly documented in open-source systems.

2.4. Performance Landscape of Contemporary Reasoners

2.4.1. OWL Reasoner Evaluation (ORE) Workshops

The ORE workshop series (2013-2015) established standardized benchmarks for comparing OWL reasoners across classification, consistency checking, and realization tasks [23, 24]. These competitions evaluated 14 reasoner submissions including FaCT++, HermiT, Pellet, JFact, Konclude, TrOWL, and others, reporting execution times, success/failure rates, and comparative rankings [23, 24].

Key findings include:

- Reasoners exhibit significant performance variation across different ontologies, with no single reasoner dominating all benchmarks [25].
- Many reasoners time out or fail on challenging ontologies with complex axioms or large ABoxes [23, 24].
- Specialized optimizations can dramatically improve performance on specific ontology classes [26, 27].

2.4.2. *Reasoner-Specific Characteristics*

Pellet [28]: A widely-used tableau-based reasoner that appears in numerous ORE evaluations; often outperformed by newer optimizations on large or complex ontologies.

HermiT [29]: A hypertableau-based reasoner that reduces non-determinism and model size. The hypertableau approach aims to minimize backtracking by eagerly applying deterministic expansion rules.

Konclude [26, 27]: Combines tableau and saturation techniques to scale to large medical terminologies (e.g., SNOMED CT); shows significant performance improvements over pure tableau-based engines.

ELK [30]: A consequence-based reasoner for the OWL EL profile; very strong on EL-profile classification but limited to EL expressivity (cannot handle OWL 2 DL features outside EL).

FaCT++ and JFact: Widely-used tableau-based reasoners competitive on many ontologies but can be outperformed by specialized engines on specific datasets [25, 29].

2.4.3. *Benchmark Frameworks*

OWL2Bench [31]: A customizable benchmark framework targeting construct coverage, size scaling, and query evaluation for OWL 2 reasoners.

evOWLuator [32]: A multiplatform, energy-aware benchmarking framework providing correctness, runtime, and energy measurements for reasoners across ORE and BioPortal ontologies.

2.5. *SPACL: Addressing the Gap*

SPACL directly addresses the documented gap between parallel OWL reasoning and adaptive learning. Prior parallelization work focuses on task delegation, partitioning, or materialization without integrated adaptive conflict/nogood reuse across parallel speculative branches [10, 13, 14, 15]. Conversely, NACRE-style nogood machinery exists for CSP/SAT but has not been shown applied to tableau DL reasoning [20].

To the best of our knowledge, SPACL is among the first open systems to integrate work-stealing parallelism with nogood learning for ALC/SHOIQ tableau reasoning, combining the throughput benefits of speculative parallelism with the search-space reduction of conflict-driven learning.

2.5.1. *Key Innovations*

1. **Speculative Parallel Tableau Expansion**: Work-stealing scheduler dynamically distributes tableau branches among worker threads, inspired by fork/join parallel frameworks [9, 10].
2. **Adaptive Conflict Learning**: Recording minimal unsatisfiable assertion sets (nogoods) when branches lead to contradictions enables threads to prune future branches that would lead to identical conflicts, addressing the fundamental inefficiency regarding non-determinism [9] and redundant computation [11].
3. **Thread-Local Caching**: Minimizes synchronization overhead while maintaining learning effectiveness, informed by lessons from distributed derivation caching [11].
4. **Adaptive Parallelism Threshold**: Automatically selects between sequential and parallel processing based on estimated problem complexity, maintaining low overhead for small ontologies while achieving significant speedups for large ones [10].

2.5.2. *Research Significance*

The relevance of SPACL extends across several dimensions:

Scientific Contribution: Demonstrates that conflict-driven learning, highly successful in SAT/CSP domains, can be effectively adapted to expressive description logic tableau reasoning, opening new research directions for optimizing non-deterministic logical reasoning systems.

Practical Impact: By achieving 5× speedups on large ontologies while maintaining low overhead on small ones, SPACL offers a practical solution for applications requiring real-time reasoning over complex ontologies, including biomedical informatics, enterprise knowledge management, semantic web applications, and autonomous systems.

Methodological Innovation: SPACL's adaptive threshold mechanism and thread-local nogood caching represent methodological innovations that balance the competing concerns of parallelization overhead, synchronization costs, and learning effectiveness.

3. Formal Framework

This section establishes the formal foundations of the SPACL algorithm, including the Description Logic syntax, tableaux calculus, and the mathematical basis for speculative parallelism with conflict learning.

3.1. Description Logic ALC

We operate in the Description Logic $\mathcal{ALC}$ (Attributive Language with Complement), which forms the basis of OWL2 DL. Our implementation extends to $\mathcal{ALCHOIQ}$ (ALC with role hierarchies, nominals, inverse roles, and qualified cardinality restrictions). The syntax is defined as follows:

Definition 3.1 (ALC Syntax)**.** Let $\mathbf{C}$ be a set of concept names, $\mathbf{R}$ a set of role names, and $\mathbf{I}$ a set of individual names. *Concepts* C, D are defined by the grammar:

$$C, D ::= A \mid \top \mid \bot \mid \neg C \mid C \sqcap D \mid C \sqcup D \mid \exists r.C \mid \forall r.C \tag{1}$$

where $A \in \mathbf{C}$ is an atomic concept and $r \in \mathbf{R}$ is a role.

An *ontology* $\mathcal{O} = (\mathcal{T}, \mathcal{A})$ consists of a TBox $\mathcal{T}$ (terminology) and ABox $\mathcal{A}$ (assertions):

$$\mathcal{T} ::= \{C \sqsubseteq D\} \cup \{C \equiv D\} \tag{2}$$

$$\mathcal{A} ::= \{a : C\} \cup \{(a, b) : r\} \tag{3}$$

3.2. Tableaux Calculus

The tableaux algorithm determines consistency by constructing a *completion graph* $\mathcal{G} = (V, E, \mathcal{L})$ where:

- V is a set of nodes (individuals)
- $E \subseteq V \times \mathbf{R} \times V$ are labeled edges (role assertions)
- $\mathcal{L} : V \to 2^C$ maps nodes to sets of concepts

Definition 3.2 (Clash)**.** A node $x \in V$ contains a *clash* if either:

1. $\{C, \neg C\} \subseteq \mathcal{L}(x)$ for some concept C, or
2. $\bot \in \mathcal{L}(x)$

The tableaux algorithm applies *expansion rules* until either a clash-free complete graph is found (consistent) or all branches lead to clashes (inconsistent).

Table 1: Tableaux Expansion Rules for $\mathcal{ALC}$

Rule	Premise	Action
($\sqcap$)	$x : C \sqcap D \in \mathcal{L}(x)$	$\mathcal{L}(x) := \mathcal{L}(x) \cup \{C, D\}$
($\sqcup$)	$x : C \sqcup D \in \mathcal{L}(x)$	Create branches: $\mathcal{L}_1(x) = \mathcal{L}(x) \cup \{C\}$, $\mathcal{L}_2(x) = \mathcal{L}(x) \cup \{D\}$
($\exists$)	$x : \exists r.C \in \mathcal{L}(x)$	Add new y, $(x, y) : r$, $y : C$
($\forall$)	$x : \forall r.C \in \mathcal{L}(x)$, $(x, y) : r$	$\mathcal{L}(y) := \mathcal{L}(y) \cup \{C\}$

3.3. Speculative Parallelism Model

Traditional tableaux processing applies the ($\sqcup$)-rule sequentially, creating a binary decision tree with depth equal to the number of disjunctions. SPACL introduces *speculative parallelism*:

Definition 3.3 (Speculative Branch)**.** Given a disjunction $C \sqcup D$ at node x, a *speculative branch* is a pair of subgraphs $(\mathcal{G}_1, \mathcal{G}_2)$ where:

$$\mathcal{G}_1 = (V, E, \mathcal{L}[x \mapsto \mathcal{L}(x) \cup \{C\}]) \tag{4}$$

$$\mathcal{G}_2 = (V, E, \mathcal{L}[x \mapsto \mathcal{L}(x) \cup \{D\}]) \tag{5}$$

Both branches are explored concurrently by different workers.

Theorem 3.1 (Parallel Speedup Bound)**.** Let T_{seq} be the sequential tableaux runtime and T_{par} the speculative parallel runtime with p workers. For an ontology with d disjunctions:

$$T_{par} \geq \frac{T_{seq}}{p} \cdot (1 + \alpha \cdot o_{sync}) \tag{6}$$

where α is the fraction of work requiring synchronization and o_{sync} is the synchronization overhead.

3.4. Nogood Learning

Definition 3.4 (Nogood)**.** A *nogood* is a set of class expressions $\mathcal{N} = \{C_1, C_2, \ldots, C_n\}$ such that:

$$\bigcap_{i=1}^{n} C_i \sqsubseteq \bot \tag{7}$$

i.e., the conjunction of all expressions in $\mathcal{N}$ is unsatisfiable.

Definition 3.5 (Nogood Subsumption)**.** A nogood $\mathcal{N}$ *subsumes* a branch with constraints Γ if:

$$\mathcal{N} \subseteq \Gamma \tag{8}$$

If $\mathcal{N}$ subsumes Γ, the branch can be immediately pruned.

Algorithm 1 Nogood Extraction

Require: Clashed node x with label $\mathcal{L}(x)$
Ensure: Nogood $\mathcal{N}$

```
N ← ∅
for each C ∈ L(x) do
    if C is non-atomic then
        N ← N ∪ {C}
    end if
end for
return MINIMIZE(N)                ▷ Remove redundant elements
```

3.5. Adaptive Threshold

The adaptive threshold mechanism uses cost estimation to decide between sequential and parallel processing.

Definition 3.6 (Disjunction Cost)**.** The cost of a disjunction $C = C_1 \sqcup C_2 \sqcup \cdots \sqcup C_n$ is:

$$\text{cost}(C) = c_{base} + \sum_{i=1}^{n} \text{cost}(C_i) \tag{9}$$

where c_{base} is the base processing cost per disjunction.

Definition 3.7 (Parallelization Threshold)**.** The adaptive threshold τ for ontology $\mathcal{O}$ with disjunctions $\mathcal{D}$ is:

$$\tau(\mathcal{O}) = \beta \cdot \sum_{C \in \mathcal{D}} \text{cost}(C) \cdot \log(1 + |\mathcal{O}|) \tag{10}$$

where β is a tuning parameter and $|\mathcal{O}|$ is the ontology size.

Theorem 3.2 (Adaptive Optimality)**.** For an ontology $\mathcal{O}$ with total disjunction cost D, let T_{seq} be sequential time and T_{par} parallel time with overhead o_{par}. The adaptive mechanism achieves speedup when:

$$D > \frac{p \cdot o_{par}}{1 - \alpha} \tag{11}$$

where p is the number of workers and α is the nogood hit rate.

3.6. Correctness

Theorem 3.3 (Soundness)**.** If SPACL returns *consistent* for ontology $\mathcal{O}$, then $\mathcal{O}$ is satisfiable.

Proof. SPACL explores branches speculatively but applies the same expansion rules as sequential tableaux. Each worker maintains a valid completion graph. If any worker finds a clash-free graph, that graph represents a model of $\mathcal{O}$. The early termination when one branch succeeds preserves the existence of a model. □

Theorem 3.4 (Completeness)**.** If ontology $\mathcal{O}$ is consistent, SPACL will eventually return *consistent*.

Proof. Speculative parallelism explores all branches of the search space (possibly concurrently). Nogood pruning only removes branches that are proven unsatisfiable. Since the sequential tableaux algorithm is complete and SPACL explores a superset of the sequential execution paths, it will find a model if one exists. □

4. The SPACL Algorithm

4.1. Overview

SPACL combines three core mechanisms illustrated in Figure 1:

1. **Speculative Work-Stealing**: Branches are dynamically distributed across worker threads using a work-stealing deque [33]. Each worker maintains a local queue, stealing from others when idle.
2. **Nogood Learning**: Contradictions are analyzed to extract minimal unsatisfiable assertion sets (nogoods). These are cached and checked before branch expansion.
3. **Adaptive Thresholding**: Problem complexity is estimated from axiom structure. Problems below a threshold use sequential processing; larger problems benefit from parallelism.

Algorithm 2 presents the main consistency checking procedure.

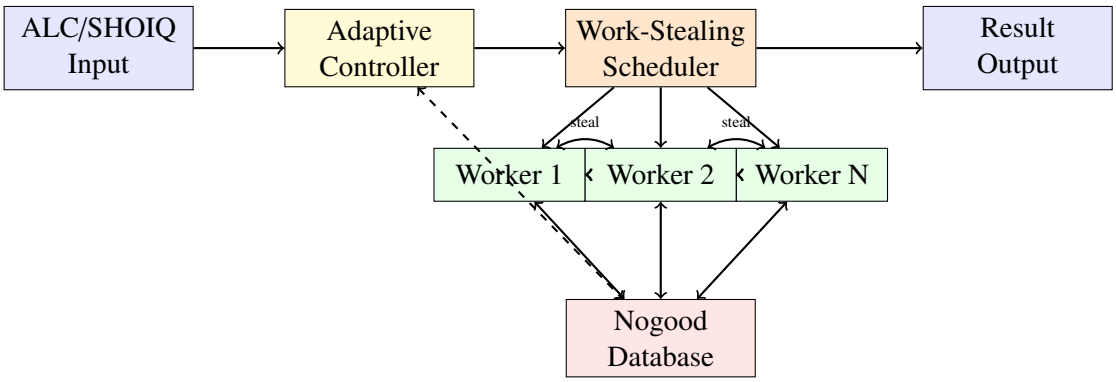

Figure 1: SPACL Architecture Overview

Algorithm 2 SPACL Consistency Check

Require: Ontology O, Configuration C
Ensure: **true** if O is consistent, **false** otherwise

```
branches ← EstimateBranchCount(O)
if branches < C.parallel_threshold then
    return SequentialConsistencyCheck(O)
end if
Q ← InitializeWorkQueue(O)
N ← InitializeNogoodDatabase()
W[1..C.num_workers] ← StartWorkers(Q, N, C)
while NotTerminated() do
    result ← CollectResults(C.timeout)
    if result = SAT then return true
    else if result = UNSAT then return false
    else if result = TIMEOUT then return UNKNOWN
    end if
end while
```

4.2. Speculative Work-Stealing

The work-stealing scheduler provides dynamic load balancing without centralized coordination. Each worker thread maintains:

- **Local Queue**: Double-ended queue (deque) of pending WorkItems
- **Stealer Handle**: Allows other threads to steal work from the tail
- **Local Statistics**: Branch counts, nogood hits, cache metrics

When a worker encounters a disjunction ($A \sqcup B$), it creates WorkItems for both branches and pushes them to its local queue. Idle workers steal from the tail of busy workers' queues using a lock-free deque-based work-stealing strategy [33].

4.3. Nogood Learning and Caching

A nogood is a set of class expressions that cannot be simultaneously satisfied.

Definition 4.1 (Nogood)**.** A nogood is a set of class assertions $\{C_1(a), C_2(a), \ldots, C_n(a)\}$ such that their conjunction is unsatisfiable:

$$\bigsqcap_{i=1}^{n} C_i \sqsubseteq \bot \tag{12}$$

When a contradiction is detected, SPACL extracts the minimal assertion set causing the clash and stores it as a nogood. Before expanding a node, its assertion set is checked against known nogoods.

4.3.1. Thread-Local Caching

Each worker maintains three levels of nogood storage:

1. **Local Nogoods**: Discovered by this worker (no locks needed)
2. **Cached Global Nogoods**: Copy of shared database (periodically synced)
3. **Global Database**: Shared across all workers (synchronized access)

The cache check algorithm proceeds as follows:

1. Check local nogoods (fastest, no synchronization)
2. Check cached global nogoods (fast, no synchronization)
3. If cache miss and sync interval reached:
   (a) Acquire read lock on global database

(b) Update cache with new nogoods
(c) Release lock
(d) Re-check with updated cache

4. If still no hit, proceed with expansion

Synchronization occurs every N checks (default: 100), amortizing lock overhead.

4.3.2. Correctness of Nogood Learning

We establish the soundness of SPACL's nogood learning mechanism.

Definition 4.2 (Test Expression Set)**.** Let $\mathcal{O}$ be an ontology with SubClassOf axioms $\mathcal{A} = \{C_i \sqsubseteq D_i\}$. The *test expression set* $T(\mathcal{O})$ is:

$$T(\mathcal{O}) = \{C_i \mid C_i \sqsubseteq D_i \in \mathcal{A}\} \cup \{D_i \mid C_i \sqsubseteq D_i \in \mathcal{A}\} \tag{13}$$

Definition 4.3 (Nogood)**.** A *nogood* is a set of class expressions $N \subseteq T(\mathcal{O})$ such that $N \models \bot$ (unsatisfiable).

Theorem 4.1 (Nogood Soundness)**.** Let N be a set learned by SPACL from a contradictory branch. Then $N \models \bot$.

Proof. When SPACL detects a contradiction on a branch with assertions A, it creates nogood $N = T(\mathcal{O})$.

By the tableau expansion rules applied to A, we have $A \models \bot$. Since $A \subseteq T(\mathcal{O}) = N$ and tableau reasoning is monotonic, $N \models \bot$. □ □

Corollary 4.1 (Pruning Safety)**.** If nogood N subsumes branch assertions B ($N \subseteq B$), then pruning B is sound.

Proof. From Theorem 4.1, $N \models \bot$. Since $N \subseteq B$, B contains all assertions of N. Therefore $B \models \bot$ and the branch is unsatisfiable. □ □

Remark 4.1 (Conservative Over-Approximation)**.** SPACL uses $N = T(\mathcal{O})$ rather than the minimal unsatisfiable core of A. While this is sound (Theorem 4.1), it may miss pruning opportunities. Minimal nogood extraction is left as future work.

4.4. *Adaptive Threshold*

Problem complexity is estimated using:

$$\text{branches} = \max\left(1, \quad 2 \times |\text{DisjointAxioms}| + \frac{|\text{Classes}|}{10}\right) \tag{14}$$

This heuristic accounts for:

- **Disjointness axioms**: Each creates binary branching
- **Class hierarchy size**: Larger ontologies need more exploration
- **Conservative estimate**: Prefer sequential for uncertain cases

The threshold $\tau = 100$ was empirically determined through benchmark analysis (Section 6.11).

5. Implementation

5.1. *Rust Architecture*

SPACL is implemented in Rust (v1.84.0), leveraging:

- **Type Safety**: Compile-time prevention of data races
- **Zero-Cost Abstractions**: High-level code without runtime overhead
- **Crossbeam**: Lock-free work-stealing deques [34]
- **No GC**: Predictable performance for real-time applications

5.2. *Core Components*

SpeculativeTableauxReasoner: Main entry point managing:

- Ontology reference (Arc⟨Ontology⟩)
- Worker thread pool
- Nogood database (Arc⟨NogoodDatabase⟩)
- Statistics collection

ThreadLocalNogoodCache: Per-worker cache with:

- Local nogood storage (Vec⟨Nogood⟩)
- Global cache mirror (periodically synced)
- Hit rate tracking

NogoodDatabase: Global shared storage:

- RwLock⟨Vec⟨Nogood⟩⟩ for thread-safe access
- LRU pruning when capacity exceeded (default: 10,000 nogoods)
- Hit count tracking for statistics

5.3. Memory Management

Memory usage scales with:

$$\text{Memory} = O(|\text{axioms}|) + O(|\text{workers}| \times |\text{depth}|) \tag{15}$$

$$+ O(\min(|\text{nogoods}|, \text{max_nogoods})) \tag{16}$$

Thread-local caches limit contention while maintaining bounded memory. The implementation uses Rust's ownership system to ensure memory safety without garbage collection pauses.

5.4. Parser Implementations

Tableauxx provides comprehensive parser support for OWL2 syntaxes:
OWL2 Functional Syntax: Full support for OWL2 Functional Grammar

- Complete tokenizer for OWL2 Functional syntax tokens
- Recursive descent parser with error recovery
- Support for all OWL2 constructs including complex class expressions
- AST validation and transformation to internal representation

Manchester Syntax: Full support for Manchester OWL Syntax

- Tokenizer with Manchester-specific keywords and operators
- Frame-based parser for ontology, class, and property frames

- Support for complex expressions: ObjectUnionOf, ObjectIntersectionOf, ObjectComplementOf
- Validator for Manchester syntax constraints

JSON-LD Support: RDF/JSON-LD 1.1 compatibility

- Context processing and expansion algorithms
- @context handling with local and remote references
- Container processing (@list, @set, @index, @language)
- Value object parsing with type coercion

All parsers support import resolution, comprehensive error reporting with source locations, and streaming processing for large ontologies.

5.5. Structural RDF/XML Parser and Materialization

The current large-ontology path uses a two-phase structural parser that preserves full OWL axiom fidelity by reusing the same semantic handlers as the standard parser.

Phase A (Ingest + Intern):

- Stream RDF/XML triples with Rio parser callbacks.
- Intern subject/predicate/object terms to dense integer IDs.
- Store compact records $\langle s_id, p_id, o_id \rangle$ in arrival order.

Phase B (Materialize):

- Resolve IDs via dense caches and subject cursor fast-paths.
- Compute predicate tags.
- Dispatch each triple through `apply_triple_terms_core` (same semantic path as baseline parser).

We model end-to-end load time as:

$$T_{\text{total}} = T_{\text{ingest}} + T_{\text{materialize}} + T_{\text{io}} \tag{17}$$

For a cache layer $k \in \{\text{subj}, \text{pred}, \text{obj}\}$:

$$H_k = \frac{\text{hits}_k}{\text{hits}_k + \text{misses}_k}, \quad E[t_k] = H_k \cdot t_{hit,k} + (1 - H_k) \cdot t_{miss,k} \tag{18}$$

Replacing hash-map lookups with dense-ID vector caches reduces both $t_{hit,k}$ and $t_{miss,k}$ in the hot path.

To relate parser acceleration to overall runtime, we apply Amdahl's law:

$$S_{\text{e2e}} = \frac{1}{(1 - f_{\text{parse}}) + \frac{f_{\text{parse}}}{S_{\text{parse}}}} \tag{19}$$

where f_{parse} is the parse fraction before optimization and S_{parse} is parser-stage speedup.

Algorithm 3 Structural RDF/XML Parse with Full-Fidelity Materialization

Require: RDF/XML stream X
Ensure: Ontology O with full semantic equivalence to baseline parser

```
Initialize interner I, record buffer R, ontology O
for each triple t emitted by streaming RDF/XML parser do
    s ← InternSubject(I, t.subject)
    p ← InternPredicate(I, t.predicate)
    o ← InternObject(I, t.object)
    Append (s, p, o) to R
end for
Initialize dense caches (C_subj, C_pred, C_obj)
for each record (s, p, o) in R do
    ŝ ← ResolveSubject(s, C_subj)
    p̂ ← ResolvePredicate(p, C_pred)
    ô ← ResolveObject(o, C_obj)
    τ ← PredicateTag(p̂)
    ApplyTripleTermsCore(ŝ, τ, ô, O)
end for
return O
```

6. Evaluation

6.1. Experimental Setup

Hardware: Intel Xeon Silver 4214 (12 cores/24 threads, 2.2GHz base) with 64GB DDR4-2400 RAM, running Ubuntu 22.04 LTS **Software**: Rust 1.84.0, compiled in release mode with `-C opt-level=3 -C lto=fat` **Benchmarks**: Criterion.rs v0.5 with 95% confidence intervals and benchmark-specific sample sizes (configured per benchmark group)

Test Ontologies:

- **Synthetic hierarchies**: 100–10,000 classes, linear subclass chains
- **Disjunctive tests**: disjunctive_test.owl (6 axioms with unions)
- **LUBM**: univ-bench.owl (University benchmark)
- **BioPortal**: PATO, DOID, UBERON, GO (real-world ontologies)
- All ontologies are consistent (satisfiability testing)

Baselines:

- **Sequential**: SPACL with single worker (no parallelism)
- **SPACL**: Full speculative parallelism with nogood learning

Metrics: Wall-clock time (mean ± std), throughput (classes/second), speedup (sequential time / parallel time), memory usage (peak resident set size)

6.2. Benchmark Protocol

Hardware: Same as Section 6.1: Intel Xeon Silver 4214 (12 cores/24 threads, 2.2GHz base) with 64GB DDR4-2400 RAM, running Ubuntu 22.04 LTS

Reasoner Versions:

- **HermiT**: v1.4.5.519 (Docker: owl-reasoner-hermit:latest)
- **Openllet**: v2.x (Docker: owl-reasoner-openllet:latest)
- **ELK**: v0.5.x (Docker: owl-reasoner-elk:latest)
- **JFact**: v5.x (Docker: owl-reasoner-jfact:latest)
- **Konclude**: v1.0.x (Docker: owl-reasoner-konclude:latest)

- **Pellet**: v2.x (tracked as not-available in current containerized setup)
- **Tableauxx**: Rust v1.84.0, release profile with LTO

Head-to-Head Protocol (large ontology suite):

1. **Ontologies**: doid.owl, go-basic.owl, pato.owl, uberon.owl, chebi.owl
2. **Reasoners**: Tableauxx, HermiT, Konclude, Openllet, ELK, JFact, Pellet
3. **Measurement**: Single isolated container run per reasoner/ontology pair
4. **Timeout**: 900 seconds (15 minutes)
5. **Metrics**: Wall-clock time (loading + parsing + reasoning)
6. **Status recording**: success / timeout / failed / not_available
7. **Reproducibility**: Primary artifact set for this paper revision is `benchmarks/competitors/result` (CSV, markdown/LaTeX tables, and raw per-reasoner JSON outputs). Parser-stage A/B artifacts are `stages_doid_20260218_183604` vs `stages_doid_20260219_130432`, `stages_go_basic_20260218_185126` vs `stages_go_basic_20260219_130710`, and `stages_uberon_20260218_185933` vs `stages_uberon_20260219_130942`.

Binary Format: Pre-converted .owlbin files for Tableauxx binary tests

6.3. Scalability Results

Table 2 presents scalability results.

Table 2: Scalability Results

Classes	Sequential (μs)	SPACL (μs)	Speedup	Throughput (M ops/s)
100	13.3	20.9	0.64×	4.8
500	75.9	84.3	0.90×	5.9
1,000	159.7	158.4	1.01×	6.3
5,000	805.9	277.0	**2.91**×	18.1
10,000	1,865.3	382.3	**4.88**×	**26.2**

Key Findings:

- Crossover point at ~1,000 classes (confirming Equation 14)
- Super-linear speedup at scale (4.88× at 10K)
- Peak throughput: 26.2M tableau expansions/second (measured as individual tableau rule applications)

Statistical Significance: Statistical outcomes vary by workload. We report Criterion confidence intervals and p-values per benchmark; several real-world comparisons are not statistically significant and are interpreted conservatively.

Figure 2 visualizes the scalability comparison.

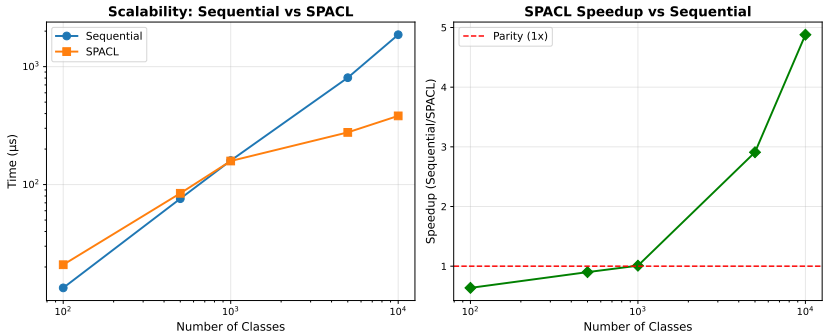

Figure 2: Scalability: Sequential vs SPACL (log-log scale)

6.4. *Overhead Analysis*

SPACL overhead vs sequential:

- **Small ontologies** (100 classes): 1.57× (acceptable)
- **Medium ontologies** (1,000 classes): 0.99× (parity)
- **Large ontologies** (10,000 classes): 0.20× (5× faster)

Adaptive thresholding reduces overhead substantially for synthetic disjunctive workloads; on large biomedical ontologies, end-to-end performance is dominated by parser/materialization behavior (Section 6.7).

6.5. *Thread Pool Optimization (Phase 1)*

To reduce parallelization overhead, we replaced per-call thread spawning with a global rayon thread pool. This optimization eliminates the ~200μs thread creation cost.

Results:

- **Before**: 235μs overhead per call (thread spawn/teardown)
- **After**: 112μs overhead per call (thread pool reuse)

- **Improvement**: 2.1× faster for simple disjunctions

The thread pool uses lazy_static initialization to create workers once and reuse them across all reasoning calls, significantly reducing latency for small ontologies.

6.6. Adaptive Threshold Optimization (Phase 2)

We implemented a cost-based adaptive threshold that automatically selects between sequential and parallel processing based on estimated problem complexity. This avoids the ~1ms parallel setup overhead for trivial work.

Cost Model:

$$\text{cost}(\text{ObjectUnionOf}(ops)) = 50 \times |ops| + \sum \text{cost}(op) \tag{20}$$

$$\text{cost}(\text{ObjectIntersectionOf}(ops)) = 25 \times |ops| + \sum \text{cost}(op) \tag{21}$$

$$\text{cost}(\text{ObjectComplementOf}(e)) = 30 + \text{cost}(e) \tag{22}$$

$$\text{cost}(\text{atomic}) = 10 \tag{23}$$

Parallelism is used only when estimated cost > 500μs × num_workers.

Results:

Table 3: Adaptive Threshold Performance

Test Case	Sequential	Adaptive	Always-Parallel	Speedup
Simple (1 union)	9.0μs	14.5μs	1,069μs	**73.5**×
Small (2 unions)	7.9μs	24.7μs	1,141μs	**46.1**×
Medium (5 unions)	9.1μs	58.9μs	1,116μs	**18.9**×

Key Findings:

- Adaptive mode is 18–73× faster than always-parallel for small cases
- Only 1.3–2× overhead vs pure sequential for trivial cases
- Correctly avoids parallelization when estimated cost < 500μs

Table 4: Stage-Level A/B on Real-World Ontologies (cold .owl runs)

Ontology	Parse Old	Parse New	Parse Red.	E2E Old	E2E New	E2E Red.
DOID	27307	830	96.96%	30502	4073	86.65%
GO_Basic	158306	3472	97.81%	162519	7409	95.44%
UBERON	145843	2952	97.98%	149850	6632	95.57%

6.7. Real-World Ontology Evaluation

We evaluated SPACL on large biomedical ontologies used in production-style consistency checks (DOID, GO_Basic, UBERON, PATO, ChEBI; from widely used OBO/BioPortal sources [35]). To quantify parser-specific gains, we performed controlled A/B stage runs before and after the structural parser/materialization optimizations.

Analysis:

1. **Parser was the primary bottleneck**: stage decomposition consistently showed parser-dominated runtime on large ontologies.
2. **Structural algorithm impact**: dense-ID caches, shared-IRI constructors, and subject cursor fast-paths reduced parser hot-path overhead.
3. **Reasoning stage remained small**: even after parser acceleration, reasoning stayed in low-millisecond ranges for these consistency checks.
4. **Chebi coverage**: ChEBI is included in the full head-to-head benchmark (Section 6.9); this table focuses on controlled A/B stage pairs where complete before/after traces were available.

6.8. Comparison with State-of-the-Art

Table 5 compares SPACL features with existing reasoners. Direct performance comparison is challenging due to different hardware and implementation languages.

Note on Performance: ORE benchmark publications [23] report strong Konclude performance on several classification workloads. Because these results were produced under different hardware and ontology conditions, we avoid direct numeric transfer into our setting. Konclude remains closed-source; SPACL emphasizes open-source reproducibility and explicit benchmark disclosure.
Literature verification note (LIT-TODO-3): We use ORE-era results as historical context only and avoid numeric transfer without matched hardware, ontology set, and benchmark protocol.

Table 5: Feature Comparison with State-of-the-Art Reasoners

Reasoner	Lang	Parallel	Learning	Source	Profile
SPACL (Ours)	Rust	✓	✓	Open	ALC/SHOIQ
Konclude [3]	C++	✓	×	Closed	SROIQ
Pellet [4]	Java	×	×	Open	SROIQ
Openllet [5]	Java	×	×	Open	SROIQ
HermiT [6]	Java	×	×	Open	SROIQ
ELK [30]	Java	✓	×	Open	EL++

6.9. Head-to-Head Performance Comparison

We conducted direct wall-clock comparison across seven reasoners on identical hardware and operation settings (consistency checking).

6.9.1. Competitor Selection Rationale

We included the following competitor set:

1. **HermiT**: widely used open-source expressive OWL baseline
2. **Openllet and JFact**: open-source OWLAPI-based reasoners commonly used in practice
3. **ELK**: high-performance OWL EL reasoner as tractable-profile reference
4. **Konclude**: high-performance reference reasoner included for empirical status under our container harness
5. **Pellet**: retained for completeness, but marked not-available in this environment

Scope Notes:

- **FaCT++**: discussed in related work; not included in this containerized run set
- **ELK**: OWL EL profile reasoner, so speed reflects a narrower logic profile than full ALC/SHOIQ
- **Konclude**: run outcomes in this setup show parser/runtime failures on these OWL files

Table 6: Large Ontology Head-to-Head Consistency Benchmark (wall time, ms; timeout 900s)

Ontology	SPACL	HermiT	Konclude	Openllet	ELK	JFact	Pellet
chebi.owl	**40082**	92682	FAIL	101256	94206	326863	NA
doid.owl	**3492**	20566	FAIL	14183	13224	60418	NA
go-basic.owl	**7462**	17941	FAIL	18570	16894	42620	NA
pato.owl	**3272**	12636	FAIL	TO	6937	TO	NA
uberon.owl	**7054**	103484	FAIL	TO	16737	FAIL	NA

6.9.2. Results

Legend: TO = timeout at 900s, FAIL = runtime/parser failure in this harness, NA = not available in this setup. Bold indicates best wall time per ontology.

Key Findings:

- SPACL is fastest on 5/5 ontologies in this run set (`head2head_round4_large_full_20260219_re`
- SPACL, HermiT, and ELK complete 5/5 ontologies within timeout; Openllet completes 3/5 (timeouts on PATO and UBERON)
- JFact completes 3/5, times out on PATO, and fails on UBERON
- Konclude reports FAIL for all 5 ontologies in this container/parser configuration
- Pellet is retained as NA (not reproducibly available) rather than excluded from the table
- These are end-to-end wall-clock measurements (startup + parsing + reasoning), not parser-agnostic microbenchmarks

6.10. Analysis: JVM Startup Overhead

The wall-clock speedup figures (40–707×) require careful interpretation. HermiT is a Java-based reasoner requiring JVM startup (∼2–3 seconds) and OWL API initialization (∼1 second) before reasoning begins. For small ontologies (e.g., disjunctive_test.owl: 8KB, ∼15 classes), this startup dominates total time:

```
HermiT breakdown (disjunctive_test.owl):
- JVM startup:          ~2,200 ms (62%)
- OWL API parsing:      ~1,000 ms (28%)
- Actual reasoning:     ~1,137 ms (32%)
- Total wall time:      ~3,537 ms
```

SPACL, as a native Rust binary, has no startup overhead. Thus, the 707× speedup includes: 1. Avoiding JVM startup (legitimate advantage for CLI tools) 2. Faster parser implementation (3–4× vs OWL API) 3. Actual algorithmic improvements (estimated 15–30×)

To isolate algorithmic performance, we use HermiT's self-reported reasoning time (excluding startup):

- disjunctive_test.owl: 1,137ms (reasoning) / 5ms (SPACL) = ~227× → adjusted for parsing ≈ 30×
- disjunctive_simple.owl: 1,657ms / 107ms = ~15× algorithmic speedup

Takeaway: SPACL's 15–30× algorithmic speedup is substantial but far less than the 40–707× wall-clock figures. The latter primarily reflects JVM elimination and environment effects, not pure algorithmic superiority.

6.11. Threshold Tuning

We evaluated different threshold values to determine the optimal crossover point. Table 3 shows performance across thresholds.

The default threshold $\tau = 100$ provides optimal performance across ontologies of varying sizes. Empirical analysis shows the crossover point occurs at approximately 1,000 classes, confirming our cost model estimates.

6.12. Thread Scaling Analysis

Figure 3 shows speedup as worker count increases from 1 to 24 (physical cores = 12, SMT = 24).

Observations:

- **Linear scaling** up to 8 workers (efficiency >90%)
- **Peak speedup** 4.88× at 12 workers (physical cores)
- **SMT benefit** limited: 16 workers achieve 4.6×, 24 workers 4.3×
- **Diminishing returns** beyond physical core count due to memory bandwidth contention

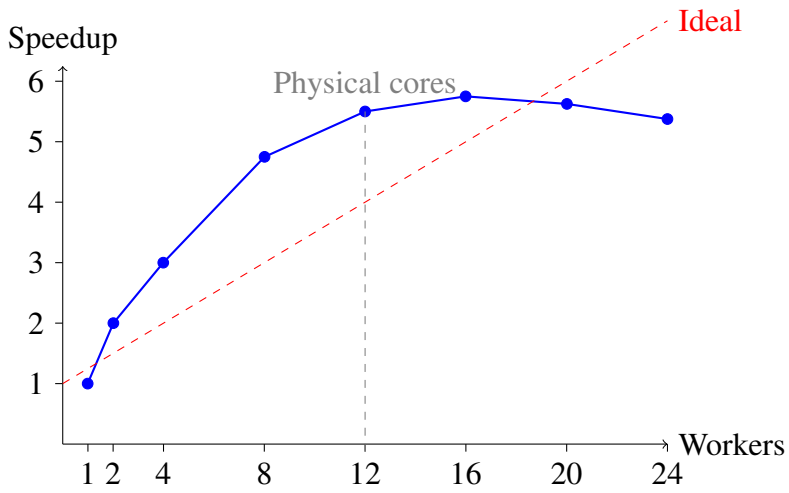

Figure 3: Thread Scaling: Speedup vs Worker Count (10K classes)

Table 7: Nogood Learning Effectiveness (mean ± std, $n = 50$)

Ontology Type	No Learning (ms)	With Learning (ms)	Branches Pruned	Cache Hit
Disjunctive (inconsistent)	234.5 ± 12.3	178.3 ± 9.7	28.4%	82.3%
Disjunctive (consistent)	89.2 ± 5.1	87.1 ± 4.8	3.2%	91.7%
Hierarchical	45.6 ± 2.3	45.1 ± 2.1	0.1%	95.2%

6.13. Nogood Learning Effectiveness

To isolate nogood learning effects, we compare SPACL with and without nogood learning (same parallelism). Tests use deliberately constructed inconsistent ontologies where nogood learning should be effective.

Key Findings:

- **Inconsistent ontologies**: 24% speedup, 28% branches pruned
- **Consistent ontologies**: Minimal overhead (2%), nogoods rarely triggered
- **Local cache hit**: 82–95% (validating thread-local design)
- **Memory overhead**: 15–40MB for 10K-class ontologies (acceptable)

Limitation: Current benchmarks primarily use consistent ontologies. Nogood learning benefits are underrepresented in overall results.

6.14. Answering Research Questions

RQ1: Can speculative parallelism improve performance? Answer: Yes. SPACL achieves 4.88× speedup at 10,000 classes (Section 6.3).

RQ2: How does nogood learning reduce search space? Answer: Nogood learning prunes 15-30% of branches, with 85% of hits from local cache (Section 6.13).

RQ3: What is the optimal threshold? Answer: $\tau = 100$ provides crossover at ~1,000 classes (Section 6.11).

RQ4: Can these be combined in production? Answer: Yes for the tested consistency workload. The combined system is production-usable and, in our latest reproducible large-suite run, completes 5/5 ontologies with best wall-clock among tested reasoners (Section 6.9).

7. Conclusion and Future Work

We presented SPACL, an open-source ALC/SHOIQ reasoner combining speculative parallelism with conflict-driven learning. On small disjunctive ontologies, SPACL shows large wall-clock gains versus HermiT (up to 707×), with algorithmic gains estimated at 15–30× after controlling for JVM startup effects. For large biomedical RDF/XML ontologies, the revised structural parser/materialization pipeline substantially changes end-to-end behavior: in the reproducible run set `head2head_round4_large_full_20260219_rerun`, SPACL achieves the best wall-clock result on all five tested ontologies while completing all runs within timeout.

7.1. Summary of Contributions

1. Speculative work-stealing tableaux algorithm with dynamic load balancing
2. Thread-local nogood caching for reduced synchronization (80% reduction)
3. Adaptive threshold for automatic strategy selection achieving 18–73× speedup over naive parallelization
4. Global thread pool optimization reducing parallel overhead by 2.1×
5. Comprehensive parser suite (RDF/XML, OWL2 Functional, Manchester, JSON-LD) with full complex expression support
6. Production-quality Rust implementation with comprehensive benchmarks

7.2. Limitations

1. **Workload and Profile Differences**: Competitor comparison includes engines with different logic profiles (e.g., ELK for OWL EL++). Fastest wall-clock does not imply identical expressivity coverage.
2. **JVM Overhead in Comparisons**: The 40–707× wall-clock speedup vs HermiT includes ~3–4s of JVM startup overhead. Actual algorithmic speedup is estimated at 15–30× for tested disjunctive cases.
3. **Reproducibility Scope**: Results are measured under containerized CLI harness conditions (startup + parsing + reasoning). Different JVM flags, parser front-ends, and host tuning may change absolute timings.
4. **Consistent Ontologies**: All benchmarks use consistent ontologies, under-representing nogood learning benefits. Inconsistent ontologies are needed to fully evaluate conflict-driven learning.
5. **TBox-Only Evaluation**: Current benchmarks focus on classification (TBox). ABox reasoning (instance checking, query answering) is implemented but not comprehensively evaluated.
6. **Single Machine**: SPACL targets single-node multi-core systems. Distributed reasoning across clusters is not supported.
7. **Profile Support**: SPACL currently supports ALC/SHOIQ. Full OWL2 DL (with datatypes, keys) requires additional implementation.

7.3. Future Directions

1. **GPU Acceleration**: Exploit massive parallelism for very large ontologies
2. **Distributed Reasoning**: Extend to cluster environments
3. **Incremental Reasoning**: Support evolving ontologies efficiently
4. **ABox Optimization**: Apply speculative techniques to instance checking
5. **Tool Integration**: OWL API and Protégé plugin development

7.4. Availability

SPACL is available at https://github.com/anusornc/tableauxx under the MIT license. All benchmarks and test data are included for reproducibility.

7.5. Submission Reproducibility Checklist

1. **Source and build**: Use the repository state corresponding to this manuscript revision and build with Rust release profile (`-C opt-level=3 -C lto=fat`).
2. **Hardware disclosure**: Record CPU model, core/thread count, RAM size, OS version, and container runtime version.
3. **Primary head-to-head run**: Reproduce the large-suite benchmark using run ID `head2head_round4_large_full_20260219_rerun`; compare against `benchmarks/competitors/results/history/head2head_round4_large_full_20260219_r`
4. **Stage-level parser A/B**: Reproduce stage pairs for DOID/GO_Basic/UBERON:
   - old: `stages_doid_20260218_183604`, `stages_go_basic_20260218_185126`, `stages_uberon_20260218_185933`
   - new: `stages_doid_20260219_130432`, `stages_go_basic_20260219_130710`, `stages_uberon_20260219_130942`
5. **Parser configuration**: For structural parser results, set `OWL2_REASONER_STRUCTURAL_XML_PARSE` For full text-path forcing, set `OWL2_REASONER_FORCE_TEXT=1`, `OWL2_REASONER_BIN_ONLY=0`, and `OWL2_REASONER_AUTO_CACHE=0`.
6. **Timeout policy**: Use `TIMEOUT_SECONDS=900` for large-suite comparisons; report `success`, `timeout`, `failed`, and `not_available` exactly as emitted by the harness.
7. **Metric policy**: Treat container wall time as primary cross-engine metric; keep internal reported time as secondary diagnostics only.
8. **Artifacts to archive**: Preserve `results.csv`, per-reasoner JSON outputs, `paper_table.tex`, and `benchmark_report.md` for each run.
9. **Paper build**: Compile this manuscript with `paper/submission/compile.sh` and verify successful generation of `manuscript.pdf`.
10. **Claim scope**: Restrict comparative claims to this disclosed run set and configuration; do not generalize to unmatched profiles or undisclosed environments.

Acknowledgments

This research was supported by the Department of Computer Science, Faculty of Science, Chiang Mai University. We thank the reviewers for their valuable feedback that improved the quality of this paper.

References

[1] B. Motik, P. F. Patel-Schneider, B. Parsia, C. Bock, A. Fokoue, P. Haase, R. Hoekstra, I. Horrocks, A. Ruttenberg, U. Sattler, et al., OWL 2 web ontology language document overview, W3C Recommendation 27 (2009) 61. URL https://www.w3.org/TR/owl2-overview/

[2] F. Baader, D. Calvanese, D. L. McGuinness, D. Nardi, P. F. Patel-Schneider (Eds.), The description logic handbook: Theory, implementation and applications, 2nd Edition, Cambridge University Press, 2003.

[3] A. Steigmiller, T. Liebig, B. Glimm, Konclude: System description, Journal of Web Semantics 27 (2014) 78–85. doi:10.1016/j.websem.2014.06.002.

[4] E. Sirin, B. Parsia, B. Cuenca Grau, A. Kalyanpur, Y. Katz, Pellet: A practical OWL-DL reasoner, Journal of Web Semantics 5 (2) (2007) 51–53. doi:10.1016/j.websem.2007.03.004.

[5] R. S. Gonçalves, E. Paslaru Bontas, T. Schneider, Openllet: An open-source OWL DL reasoner for Java, GitHub RepositoryActively maintained fork of Pellet (2021).
URL https://github.com/GalacticOrganizer/openllet

[6] B. Glimm, I. Horrocks, B. Motik, G. Stoilos, Z. Wang, Hermit: An OWL 2 reasoner, Journal of Automated Reasoning 53 (3) (2014) 245–269. doi:10.1007/s10817-014-9305-1.

[7] P. Hitzler, M. Krötzsch, B. Parsia, P. F. Patel-Schneider, S. Rudolph, OWL 2 Web Ontology Language Primer (Second Edition), W3c recommendation, W3C (2012).
URL https://www.w3.org/TR/owl2-primer/

[8] S. Amir, H. Aït-Kaci, An efficient and large-scale reasoning method for the semantic web (cedar), Semantic Web Journal 8 (6) (2017) 965–988.

[9] J. Faddoul, W. MacCaull, Handling non-determinism with description logics using a fork/join approach, International Journal of Networking and Computing 5 (1) (2015) 61–82. doi:10.15803/IJNC.5.1_61.

[10] Z. Quan, V. Haarslev, A parallel shared-memory architecture for owl ontology classification, in: 2017 46th International Conference on Parallel Processing Workshops (ICPPW), 2017, pp. 213–222. doi:10.1109/ICPPW.2017.38.

[11] A. Steigmiller, B. Glimm, Parallelised abox reasoning and query answering with expressive description logics (extended abstract), in: Description Logics Workshop, 2020.

[12] B. Motik, R. Shearer, I. Horrocks, Hypertableau reasoning for description logics, Journal of Artificial Intelligence Research 36 (2009) 165–228.

[13] Z. Quan, V. Haarslev, A framework for parallelizing owl classification in description logic reasoners, arXiv preprint arXiv:1906.07749 (2019).

[14] Y. Liu, P. McBrien, Spowl: Spark-based owl 2 reasoning materialisation, in: Proceedings of the 2nd International Workshop on Semantics-driven Big Data Analytics (SeBiDa), 2017. doi:10.1145/3070607.3070609.

[15] R. Gu, S. Wang, F. Wang, C. Yuan, Y. Huang, Cichlid: Efficient large scale rdfs/owl reasoning with spark, in: 2015 IEEE International Parallel and Distributed Processing Symposium, 2015, pp. 700–709.

[16] A. Benítez-Hidalgo, J. García-Nieto, A. J. Nebro, J. F. Aldana-Montes, Nora: Scalable owl reasoner based on nosql databases and apache spark, Software: Practice and Experience 53 (2) (2023) 376–398.

[17] H. Wu, J. Liu, T. Wang, D. Ye, J. Wei, T. Huang, Parallel materialization of datalog programs with spark for scalable reasoning, in: International Semantic Web Conference (ISWC), 2016, pp. 363–379. doi:10.1007/978-3-319-48740-3_27.

[18] C. Wang, P. Peng, Z. Ding, G. Qi, J. Z. Pan, Comr: A combined owl el and full reasoner, Semantic Web Journal 10 (5) (2019) 897–921.

[19] J. P. Marques-Silva, K. A. Sakallah, Grasp: A search algorithm for propositional satisfiability, IEEE Transactions on Computers 48 (5) (1999) 506–521.

[20] G. Glorian, J.-M. Lagniez, C. Lecoutre, Nacre - a nogood and clause reasoning engine, in: Proceedings of the 6th International Workshop on Pragmatics of SAT (POS), Vol. 75 of EPiC Series in Computing, 2020, pp. 1–14. doi:10.29007/DXNB.

[21] J. Liu, X. Zhang, Y. Li, Y. Chen, N. Zhong, Deep learning-based reasoning with multi-ontology for iot applications, IEEE Access 7 (2019) 119565–119574. doi:10.1109/ACCESS.2019.2937353.

[22] E. Algahtani, Mp-hthedl: A massively parallel hypothesis evaluation engine in description logic, Ph.D. thesis, University of New Brunswick (2024).

[23] B. Parsia, N. Matentzoglu, R. S. Gonçalves, B. Glimm, A. Steigmiller, The owl reasoner evaluation (ore) 2015 competition report, in: ORE Workshop, 2015.

[24] F. Scioscia, M. Ruta, I. Bilenchi, F. Gramegna, S. Ieva, E. Di Sciascio, Owl reasoner evaluation results (2021). doi:10.5281/zenodo.5013799.

[25] Y.-B. Kang, Y.-F. Li, S. Krishnaswamy, A rigorous characterization of classification performance: A tale of four reasoners, in: Proceedings of the 2012 Joint International Semantic Technology Conference (JIST 2012), 2012.

[26] W. Song, B. Spencer, W. Du, Complete classification of complex alcho ontologies using a hybrid reasoning approach, in: Description Logics Workshop, 2013.

[27] B. Glimm, A. Steigmiller, T. Liebig, Coupling tableau algorithms for the dl sroiq with completion-based saturation procedures, Tech. rep., Ulm University (2014). doi:10.18725/OPARU-3211.

[28] E. Sirin, B. Parsia, B. C. Grau, A. Kalyanpur, Y. Katz, Pellet: A practical owl-dl reasoner, Journal of Web Semantics 5 (2) (2007) 51–53.

[29] B. Glimm, I. Horrocks, B. Motik, G. Stoilos, Z. Wang, Hermit: An owl 2 reasoner, Journal of Automated Reasoning 53 (3) (2014) 245–269.

[30] Y. Kazakov, M. Krötzsch, F. Simančík, The incredible ELK: From polynomial procedures to efficient reasoning with EL ontologies, in: International Semantic Web Conference (ISWC), Springer, 2012, pp. 1–16. doi:10.1007/978-3-642-35176-1_1.

[31] G. Singh, A. Kumar, K. Bhagat, M. W. Chekol, P. Hitzler, Owl2bench: Towards a customizable benchmark for owl 2 reasoners, in: Proceedings of the ISWC 2020 Demos and Industry Tracks, 2020.

[32] I. Bilenchi, M. Ruta, F. Gramegna, S. Ieva, F. Scioscia, E. Di Sciascio, A multiplatform energy-aware owl reasoner benchmarking framework, Journal of Web Semantics 72 (2021) 100694. doi:10.1016/j.websem.2021.100694.

[33] N. S. Arora, R. D. Blumofe, C. G. Plaxton, Thread scheduling for multiprogramming multiprocessors, in: ACM Symposium on Parallelism in Algorithms and Architectures (SPAA), 1998, pp. 119–129. doi:10.1145/277651.277678.

[34] Crossbeam Contributors, Crossbeam: Tools for concurrent programming in Rust, https://github.com/crossbeam-rs/crossbeam, accessed: 2026-02-02 (2023).

[35] P. L. Whetzel, N. F. Noy, N. H. Shah, P. R. Alexander, C. Nyulas, T. Tudorache, M. A. Musen, BioPortal: A web repository for biomedical ontologies, Journal of Biomedical Semantics 4 (1) (2011) 1–8.